DAY 5

Build, Deploy & AI Hackathon

Theme: From Python Code to a Real AI Product

Complete Study Guide: concepts, definitions, diagrams, code snippets and sample outputs

- Streamlit application development (pages, inputs, buttons, uploads, sidebar, tables, charts, AI responses)
- AI application architecture and separation of logic and secrets
- Security basics: environment variables, .env files, why keys are never hard-coded
- Git and GitHub workflow, README and portfolio presentation
- Free deployment: Streamlit Community Cloud, Render, Hugging Face Spaces
- Final AI Hackathon workflow and four hands-on projects

> **KEY CONCEPT**
> Code in this guide is deliberately **concept-focused**: short snippets that show the idea behind each feature, not full applications. Once you understand the concept, you can assemble the pieces into your own project. Sample outputs are illustrative; your AI model's exact wording will differ.

Contents

1. Day 5 Overview: The Big Picture 3

2. Streamlit Application Development 4

3. AI Application Architecture 9

4. Security Basics 12

5. Git and GitHub 14

6. Deployment 16

7. Final AI Hackathon 18

8. Hands-On Projects and Challenges 20

9. Quick Revision Cheat Sheet 25

1. Day 5 Overview: The Big Picture

Until now, your AI work lived inside notebooks and scripts that only you could run. Day 5 is about crossing the gap between **code** and **product**: a real application that someone else can open in a browser, use, and be impressed by.

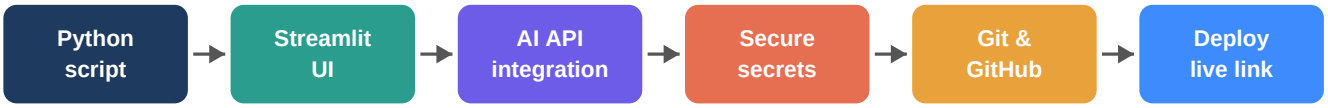


Figure 1: The journey from Python code to a live AI product

What turns code into a product?

Ingredient	Meaning	Tool used today
Interface	A UI that non-programmers can use	Streamlit
Intelligence	A model that generates, analyses or explains	LLM API (OpenAI, Gemini, Groq, etc.)
Safety	Secrets protected, errors handled	.env, st.secrets, .gitignore
Versioning	History of your work, shareable code	Git and GitHub
Availability	A public URL anyone can open	Streamlit Cloud / Render / HF Spaces
Story	README, demo, presentation	Markdown README + live demo

Day 5 learning outcomes

- Build a multi-feature web app in pure Python using Streamlit.
- Explain how UI, logic and AI API are separated, and why.
- Store and load API keys safely without exposing them.
- Use the core Git commands and publish a project on GitHub.
- Deploy the app for free and share a working link.
- Plan, build, test, document, deploy and present within a hackathon timeline.

2. Streamlit Application Development

2.1 What is Streamlit?

Definition: Streamlit is an open-source Python framework that turns a Python script into an interactive web application. You do not write HTML, CSS or JavaScript; every widget (text box, button, chart) is a single Python function call.

- **Why it suits AI apps:** data scientists can build a UI in minutes using the libraries they already know (pandas, matplotlib, scikit-learn, LLM SDKs).
- **Run command:** `streamlit run app.py`, which opens the app at `http://localhost:8501`.

Terminal

```
pip install streamlit
streamlit run app.py
```

Output

```
  You can now view your Streamlit app in your browser.

  Local URL:   http://localhost:8501
  Network URL: http://192.168.1.12:8501
```

2.2 How Streamlit works: the rerun model

This is the single most important Streamlit concept. **Every time the user interacts with a widget, Streamlit re-runs your entire script from top to bottom** and redraws the page. Widgets simply return their current values during that run.

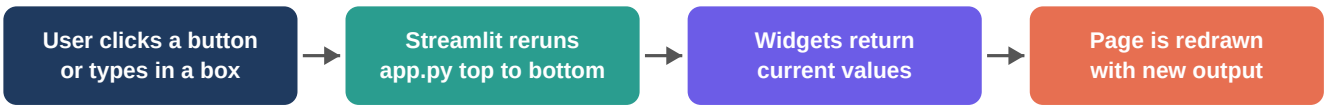


Figure 2: Streamlit's rerun cycle

> **KEY CONCEPT**
> Because the script reruns on every interaction, normal variables are reset each time. To remember something between reruns (chat history, quiz score, generated questions) store it in `st.session_state`. To avoid repeating slow work (loading a file, calling an API with the same input) use `@st.cache_data`.

2.3 Page structure

A Streamlit page is built from top-level text elements and layout containers. Page-level options are set once, at the very top, using `st.set_page_config()`.

Element	Purpose
`st.title / header / subheader`	Headings of decreasing size
`st.write / st.markdown`	General text, Markdown, variables, DataFrames
`st.caption`	Small, grey helper text
`st.columns(n)`	Side-by-side layout
`st.tabs([...])`	Tabbed sections (ideal for multi-feature apps)

Element	Purpose
`st.expander`	Collapsible section
`st.divider`	Horizontal line separator

Code

```
import streamlit as st

st.set_page_config(page_title="AI Study Buddy", layout="wide")   # must be first st command

st.title("AI Study Buddy")
st.caption("Ask. Learn. Revise.")

tab1, tab2 = st.tabs(["Summary", "Quiz"])
with tab1:
    st.subheader("Summarise your notes")
with tab2:
    st.subheader("Practise MCQs")

col_in, col_out = st.columns(2)
col_in.write("Input goes here")
col_out.write("Output goes here")
```

Output (what the user sees)

```
[Browser view]
 AI Study Buddy
 Ask. Learn. Revise.
 [ Summary ] [ Quiz ]          <- clickable tabs
 Summarise your notes
 | Input goes here  | Output goes here |   <- two equal columns
```

2.4 Text inputs

Input widgets collect data from the user. Each one is a function that **returns the current value**: an empty string for text boxes before typing, or the selected option for choice widgets.

Widget	Returns	Typical AI-app use
`st.text_input`	str (one line)	Topic, job role, name
`st.text_area`	str (multi-line)	Paste notes, resume text, expenses
`st.number_input`	int / float	Monthly income, number of questions
`st.selectbox`	selected item	Difficulty, interview type
`st.slider`	number	Creativity (temperature), question count
`st.chat_input`	str or None	Chatbot message box

Code

```
role  = st.text_input("Target job role", placeholder="e.g. Data Analyst")
notes = st.text_area("Paste your notes", height=150)
level = st.selectbox("Difficulty", ["Easy", "Medium", "Hard"])
count = st.slider("Number of questions", min_value=1, max_value=10, value=5)

st.write(f"You chose {count} {level} questions for {role}")
```

Output

```
You chose 5 Medium questions for Data Analyst
```

2.5 Buttons

`st.button()` returns **True only on the run triggered by the click**, then goes back to False. That is why AI calls are placed inside an `if st.button(...):` block: the (expensive) API call happens only when the user asks for it, not on every rerun.

Code

```
if st.button("Generate Summary"):
    if not notes.strip():
        st.warning("Please paste some notes first.")      # validate input
    else:
        with st.spinner("Thinking..."):                   # loading animation
            result = generate_summary(notes)              # calls the AI layer
        st.success("Done!")
        st.write(result)
```

Output

```
[Before click]  (nothing shown)
[While running] (spinner) Thinking...
[After]         Done!  <- green box
                "Photosynthesis is the process by which plants convert light..."
```

> **KEY CONCEPT**
> Status messages: `st.success` (green), `st.info` (blue), `st.warning` (yellow), `st.error` (red). Always validate input before calling an AI API so you do not waste tokens or money on empty requests.

2.6 File upload

`st.file_uploader()` lets users upload files (PDF, TXT, CSV). It returns **None** until a file is chosen, and then a file-like object. The AI model cannot read a PDF directly, so the typical pipeline is: **upload, extract text, send the text to the model**.

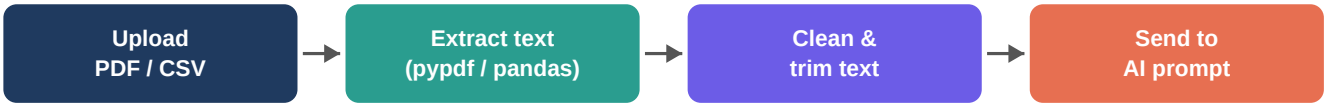


Figure 3: File-upload pipeline

Code

```
from pypdf import PdfReader

file = st.file_uploader("Upload your resume", type=["pdf", "txt"])

if file is not None:
    if file.name.endswith(".pdf"):
        reader = PdfReader(file)
        text = "\n".join(page.extract_text() or "" for page in reader.pages)
    else:
        text = file.read().decode("utf-8")

    st.success(f"Read {len(text)} characters from {file.name}")
    with st.expander("Preview extracted text"):
        st.text(text[:500])
```

Output

```
Read 3142 characters from asha_resume.pdf
> Preview extracted text
  ASHA REDDY  |  Final-year CSE (AI & DS)
  Skills: Python, SQL, Pandas, Scikit-learn ...
```

2.7 Sidebar

The sidebar is a collapsible panel on the left, ideal for **settings and navigation** so the main area stays clean. Use `with st.sidebar:` or `st.sidebar.`.

Code

```
with st.sidebar:
    st.header("Settings")
    feature = st.radio("Choose feature", ["Summary", "MCQs", "Study Plan"])
    temperature = st.slider("Creativity", 0.0, 1.0, 0.3)
    st.caption("Lower = factual, higher = creative")

if feature == "Summary":
    ...   # show the summary UI
elif feature == "MCQs":
    ...   # show the quiz UI
```

Output (layout)

```
 +-----------------+---------------------------+
 | Settings        |  (main area changes based  |
 | (o) Summary     |   on the selected feature) |
 | ( ) MCQs        |                           |
 | ( ) Study Plan  |                           |
 | Creativity 0.3  |                           |
 +-----------------+---------------------------+
```

2.8 Tables

Function	Behaviour	Use when
`st.dataframe(df)`	Interactive: sortable, scrollable	Large or exploratory data
`st.table(df)`	Static, fully displayed	Small result tables
`st.data_editor(df)`	User can edit cells	Editable expense entries

Code

```
import pandas as pd

df = pd.DataFrame({
    "Skill":  ["Python", "SQL", "Docker"],
    "Status": ["Strong", "Medium", "Missing"],
})
st.dataframe(df, use_container_width=True)
```

Output

```
   Skill    Status
0  Python   Strong
1  SQL      Medium
2  Docker   Missing
```

2.9 Charts

Streamlit has built-in one-line charts (`bar_chart, line_chart, area_chart`). For more control, use Matplotlib, Plotly or Altair with `st.pyplot` / `st.plotly_chart`. `st.metric` displays a headline number with a change indicator, useful for dashboards.

Code

```
spend = pd.DataFrame({"Amount": [8000, 4500, 3000, 1500]},
                     index=["Rent", "Food", "Travel", "Fun"])

st.metric("Total Spent", "Rs. 17,000", delta="-8% vs last month")
st.bar_chart(spend)
```

Output (rendered as an interactive bar chart)

```
 Total Spent
 Rs. 17,000    (down) -8% vs last month

 Rent   |#########################  8000
 Food   |###############            4500
 Travel |##########                 3000
 Fun    |#####                      1500
```

2.10 AI responses (chat style)

AI apps typically show responses in one of two styles: a **single result** (button, then answer) or a **conversation** (chat). For conversations, history must live in `st.session_state` because the script reruns after every message.



Figure 4: Chat loop with memory

Code

```
if "messages" not in st.session_state:
    st.session_state.messages = []                      # create once

for m in st.session_state.messages:                     # replay history
    with st.chat_message(m["role"]):
        st.markdown(m["content"])

if prompt := st.chat_input("Ask me anything..."):
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    reply = ask_llm(prompt)                             # AI layer (Section 3)
    with st.chat_message("assistant"):
        st.markdown(reply)
    st.session_state.messages.append({"role": "assistant", "content": reply})
```

Output

```
 [user]      Explain overfitting in one line.
 [assistant] Overfitting is when a model memorises training data, noise included,
             and therefore performs poorly on new, unseen data.
```

3. AI Application Architecture

3.1 The core idea: UI, logic, AI API, response

Every AI app, from a simple summariser to a full product, follows the same pipeline. The user never talks to the model directly: your Python code sits in the middle, shaping the request and presenting the response.

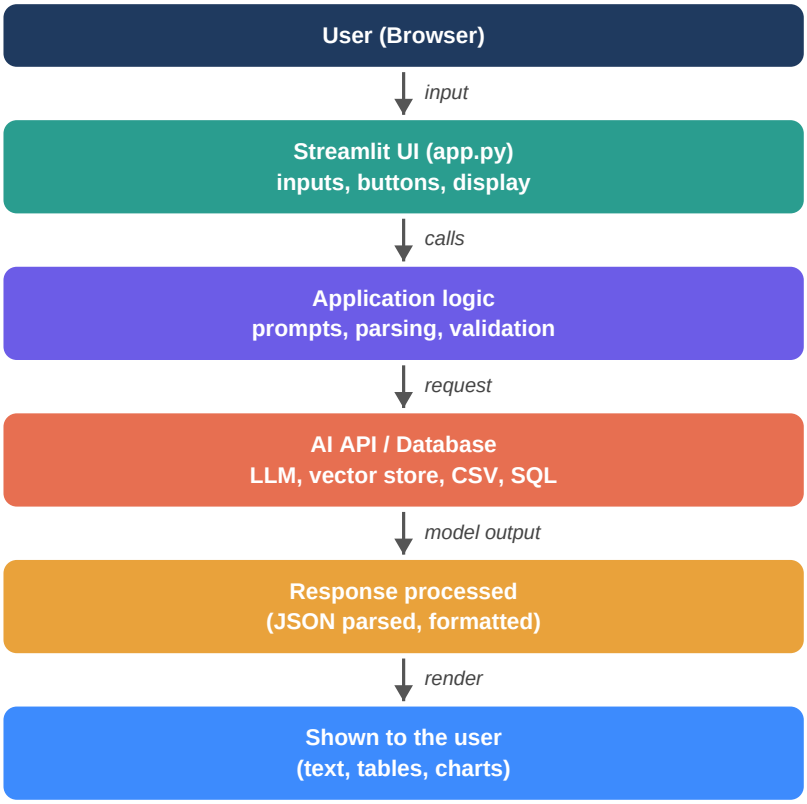


Figure 5: Layered AI application architecture

3.2 What each layer does

Layer	Responsibility	Lives in
UI layer	Collect input, show output. No business rules or API calls.	app.py
Logic layer	Build prompts, validate input, parse the model's JSON, handle errors.	utils.py, prompts.py
AI / data layer	Talk to the LLM API, database, vector store or files.	llm.py
Config / secrets	API keys and settings, kept outside the code.	.env / st.secrets

3.3 Separation of application logic and secrets

Separation of concerns means each file has one job. Benefits: you can change the UI without touching AI code, switch from one AI provider to another by editing only `llm.py`, test each part independently, and keep secrets out of any file that goes to GitHub.

Recommended project structure

```
ai-app/
|-- app.py              # Streamlit UI only
|-- llm.py              # one function to call the AI model
|-- prompts.py          # all prompt templates in one place
|-- utils.py            # PDF reading, JSON parsing, helpers
|-- requirements.txt    # dependencies for deployment
|-- README.md           # project documentation
|-- .env                # SECRET keys (never committed)
|-- .gitignore          # lists .env so Git ignores it
```

llm.py

```
# llm.py : the ONLY file that knows about the AI provider
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()                                        # reads .env into environment
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def ask_llm(system_prompt, user_prompt, temperature=0.3):
    response = client.chat.completions.create(
        model="gpt-4o-mini",                         # swap model/provider here only
        temperature=temperature,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user",   "content": user_prompt},
        ],
    )
    return response.choices[0].message.content
```

app.py

```
# app.py : UI only, imports the AI function
import streamlit as st
from llm import ask_llm
from prompts import SUMMARY_PROMPT

text = st.text_area("Paste text")
if st.button("Summarise"):
    st.write(ask_llm("You are a helpful tutor.", SUMMARY_PROMPT.format(text=text)))
```

> **KEY CONCEPT**
> The same structure works with any provider (OpenAI, Google Gemini, Groq, Anthropic, Hugging Face, Ollama). Only the body of `ask_llm()` changes; the rest of your app is untouched.

3.4 Prompt design: the logic layer's main job

A prompt is the program you write for the model. Reliable prompts contain five parts:

Part	Example
Role	You are an expert technical recruiter.
Task	Analyse the resume for the target role.
Context / data	Resume text and role inserted via placeholders.
Output format	Return ONLY valid JSON with keys: skills, gaps, suggestions.
Constraints	Max 5 suggestions. Be concise. Do not invent experience.

> **WARNING**
> Models sometimes wrap JSON in extra text or code fences. Always wrap `json.loads()` in try/except, strip stray backticks, and show a friendly error instead of letting the app crash.

4. Security Basics

4.1 What is an API key and why does it matter?

An **API key** is a secret password that identifies you to an AI provider and lets you use (and be billed for) their service. Anyone who has your key can make requests as you, using your quota or money.

4.2 Why API keys should never be hard-coded

Wrong

```
# BAD: key written directly in source code
client = OpenAI(api_key="sk-abc123-my-real-secret-key")
```

- **GitHub leaks:** once code is pushed, the key is in the repository history. Automated bots scan public repositories and can find leaked keys within minutes.
- **Financial risk:** strangers can run up charges or exhaust your free quota.
- **Hard to rotate:** changing the key means editing and redeploying code.
- **Team risk:** everyone with the code has the secret.

4.3 Environment variables

Definition: an environment variable is a named value stored in the operating system's environment (outside your code) that a program can read at run time. Your code asks for `OPENAI_API_KEY` by name; the value is supplied separately on each machine.

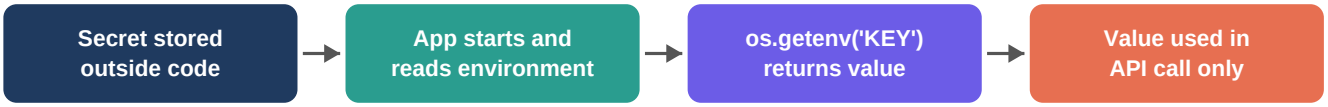


Figure 6: How secrets reach your program without living in your code

4.4 The .env file concept

Setting environment variables manually is tedious. A **.env file** is a plain-text file of `NAME=value` lines kept in your project folder. The `python-dotenv` library loads it into the environment when your app starts. The file stays on your computer and is **never committed to Git**.

.env file

```
# .env  (stays on your machine)
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxx
```

.gitignore file

```
# .gitignore  (tells Git to skip these)
.env
venv/
__pycache__/
.streamlit/secrets.toml
```

Correct way

```
import os
from dotenv import load_dotenv

load_dotenv()                              # loads .env into environment variables
api_key = os.getenv("OPENAI_API_KEY")      # returns None if missing

if not api_key:
    st.error("API key not found. Add OPENAI_API_KEY to your .env file.")
    st.stop()                              # stop the app gracefully
```

4.5 Secrets on deployed platforms

The .env file is not uploaded, so cloud platforms give you a **Secrets / Environment Variables** settings page where you paste the same names and values. Locally you use .env; in the cloud you use the platform's secret store.

Code

```
# Works both locally and on Streamlit Cloud
def get_api_key():
    try:
        return st.secrets["OPENAI_API_KEY"]     # Streamlit Cloud secrets
    except Exception:
        return os.getenv("OPENAI_API_KEY")      # local .env
```

4.6 Security checklist

Do	Do not
Store keys in .env or platform secrets	Paste keys into code, notebooks or screenshots
Add .env to .gitignore BEFORE the first commit	Commit .env 'just once'; it stays in history
Revoke and regenerate a key immediately if it leaks	Assume a deleted commit is safe
Set usage limits on your API account	Share keys over chat or email
Provide a .env.example with fake placeholder values	Show real keys in the README

5. Git and GitHub

5.1 Definitions

- **Git:** a version-control system that records snapshots (commits) of your project so you can track changes, go back in time, and collaborate.
- **GitHub:** a website that hosts Git repositories online. It is your portfolio, your backup and the source from which most free platforms deploy your app.
- **Repository (repo):** a project folder tracked by Git.
- **Commit:** a saved snapshot with a message describing what changed.

5.2 The three areas of Git



Figure 7: How changes travel from your editor to GitHub

5.3 The core commands

Command	What it does	Analogy
`git init`	Starts tracking the current folder as a repo (creates hidden .git folder)	Open a new notebook
`git status`	Shows modified, staged and untracked files	Check what changed
`git add .`	Moves all changes into the staging area (or name specific files)	Put items in the box
`git commit -m 'msg'`	Saves the staged snapshot permanently with a message	Seal and label the box
`git remote add origin URL`	Links the local repo to a GitHub repo	Save the delivery address
`git push -u origin main`	Uploads commits to GitHub	Send the box
`git pull`	Downloads new commits from GitHub	Receive updates

Typical first-time session

```
$ git init
Initialized empty Git repository in /home/asha/ai-app/.git/

$ git add .
$ git commit -m "Initial commit: Streamlit AI resume analyzer"
[main (root-commit) 3f2a1bc] Initial commit: Streamlit AI resume analyzer
 6 files changed, 142 insertions(+)

$ git branch -M main
$ git remote add origin https://github.com/<username>/ai-resume-analyzer.git
$ git push -u origin main
Enumerating objects: 9, done.
To https://github.com/<username>/ai-resume-analyzer.git
 * [new branch]      main -> main
```

> **KEY CONCEPT**
> Before your first **git add**, confirm `.gitignore` contains `.env`. Run `git status` and make sure .env does not appear in the list of files to be committed.

Good commit messages

- Write in the present tense and be specific: `Add PDF upload and text extraction`.
- Avoid vague messages such as `update` or `final final 2`.
- Commit small and often: each commit should represent one logical change.

5.4 Project README: your project's front page

Recruiters and judges often read only your README. It should answer: what is it, why does it matter, how do I use it, and how do I run it?

README.md template

```
# AI Resume Analyzer

Upload a resume and get skills, gaps, suggestions and interview questions.

**Live demo:** https://your-app.streamlit.app
**Screenshot:** ![demo](assets/demo.png)

## Features
- PDF resume upload and text extraction
- AI-generated skill gap analysis for a chosen role
- Tailored interview questions

## Tech Stack
Python, Streamlit, OpenAI API, pypdf, pandas

## How It Works
UI (Streamlit) -> logic (prompts + parsing) -> LLM API -> formatted results

## Run Locally
1. git clone https://github.com/<username>/ai-resume-analyzer.git
2. pip install -r requirements.txt
3. Create .env with OPENAI_API_KEY=your_key
4. streamlit run app.py

## Future Improvements
- Job-description matching, multi-resume comparison

## Author
Your Name | LinkedIn | GitHub
```

5.5 Portfolio presentation

- **Pin** your best 2-4 repositories on your GitHub profile.
- Add a **live link**, a clear **screenshot or GIF**, and a one-line description at the top of every README.
- Explain the **problem, your solution, tech stack and what you learned**; focus on impact, not just tools.
- Keep the repo tidy: clear folder structure, requirements.txt, no secrets, no giant unused files.
- Link the project in your resume and LinkedIn. A working demo beats a bullet point.

6. Deployment

6.1 What is deployment?

Definition: deployment is making your application run on a server connected to the internet so anyone can use it through a URL, without installing Python or running code themselves.

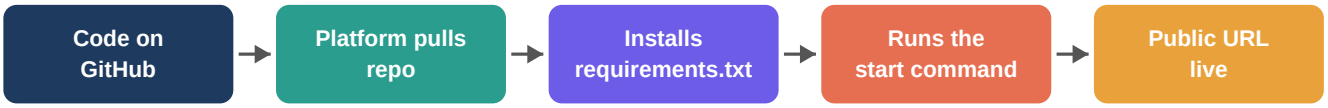


Figure 8: What a deployment platform does for you

6.2 Before you deploy: checklist

- **requirements.txt** lists every library (platforms install from it).
- Entry file is clear (usually `app.py`).
- No secrets in the code or repo; keys will be added in the platform's settings.
- The app runs locally without errors.

requirements.txt

```
streamlit
openai
python-dotenv
pypdf
pandas
```

6.3 Free deployment options compared

Platform	Best for	How you deploy	Secrets go in
Streamlit Community Cloud	Streamlit apps; fastest path	Sign in with GitHub, choose repo, branch and main file, click Deploy	App settings, Secrets (TOML format)
Render	General web apps and APIs	Create a Web Service from the repo, set build and start commands	Environment tab
Hugging Face Spaces	ML/AI demos and community visibility	Create a Space, choose the Streamlit SDK, push files	Space Settings, Variables and secrets

PRACTICAL TIP

Free tiers have limits (for example apps may go to sleep when idle, and resources are modest). Check each platform's current free-tier terms before relying on it for a demo, and open your link once before presenting to wake it up.

6.4 Streamlit Community Cloud (steps)

- 1. Push your project to a **public GitHub repo** (no .env inside).
- 2. Go to share.streamlit.io and sign in with GitHub.
- 3. Click **New app**, then pick repository, branch `main` and main file `app.py`.
- 4. Open **Advanced settings, Secrets** and paste your keys in TOML format (below).
- 5. Click **Deploy**. You receive a URL like `https://yourapp.streamlit.app`.

Secrets

```
# Secrets box on Streamlit Cloud (TOML)
OPENAI_API_KEY = "sk-xxxxxxxxxxxxxxxx"
```

6.5 Render (key settings)

Render web service

```
Build Command:  pip install -r requirements.txt
Start Command:  streamlit run app.py --server.port $PORT --server.address 0.0.0.0
Env Variable:   OPENAI_API_KEY = sk-xxxxxxxx
```

6.6 Hugging Face Spaces (key settings)

Create a new Space, select **Streamlit** as the SDK, then upload `app.py` and `requirements.txt`. The Space's README.md begins with a small configuration header:

README.md header in a Space

```
---
title: AI Study Buddy
emoji: books
sdk: streamlit
app_file: app.py
---
```

6.7 Common deployment errors

Symptom	Likely cause	Fix
ModuleNotFoundError	Library missing from requirements.txt	Add it and redeploy
Key not found / authentication error	Secret not added in platform settings	Add secret with exact same name
App works locally, fails online	Hard-coded local file paths	Use relative paths
Blank page / port error (Render)	Wrong start command	Use --server.port $PORT and --server.address 0.0.0.0

7. Final AI Hackathon

The hackathon is your capstone for the course: a compact, end-to-end experience of building and shipping a real AI product under time pressure, exactly as in industry.

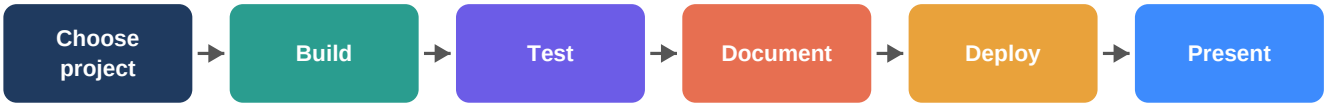


Figure 9: Hackathon workflow

7.1 Stage by stage

Stage	What to do	Output
1. Choose	Pick one of the four projects (or your own). Define user, problem, and a one-sentence value statement.	Scope you can finish
2. Build	Start with the simplest working loop (input, AI call, output). Then add features one at a time.	Working Streamlit app
3. Test	Try empty input, very long input, wrong file type, API failure. Fix crashes with friendly messages.	Stable app
4. Document	Write README (features, tech, run steps, screenshot). Add requirements.txt and .env.example.	Complete repository
5. Deploy	Push to GitHub, deploy to a free platform, add secrets, test the public link.	Live URL
6. Present	Tell the story: problem, demo, how it works, what you learned, what is next.	Confident demo

7.2 Suggested time split

Phase	Share of time	Tip
Plan and set up	10%	Folder structure, .gitignore, API key working
Build core feature	40%	Get one feature fully working first
Polish and extra features	20%	Sidebar, charts, error messages
Test and document	15%	README and screenshots
Deploy and rehearse	15%	Deploy early, not in the last minute

7.3 What judges usually look for

- **Problem relevance:** does it solve a real, clearly stated problem?
- **Working demo:** does it run live without crashing?
- **AI use:** is the model used meaningfully (good prompts, structured output), not just a thin wrapper?
- **Engineering quality:** clean structure, secrets handled safely, error handling.
- **Presentation:** clear story, readable README, working public link.

7.4 Five-minute presentation outline

Minute	Content
0:00 - 0:45	Problem and who it is for
0:45 - 2:45	Live demo of the main flow (prepare a sample input in advance)
2:45 - 3:45	Architecture diagram and tech stack: UI, logic, AI API, secrets, deployment
3:45 - 4:30	Challenges faced and what you learned
4:30 - 5:00	Future improvements and the live link / QR code

PRESENTATION TIP

Always have a **backup**: screenshots or a short screen recording in case the internet or the API fails during the demo.

8. Hands-On Projects and Challenges

All four projects share the same skeleton: **input, prompt, AI call, parse, display**. What changes is the input type and the prompt. Learn the pattern once; reuse it everywhere.

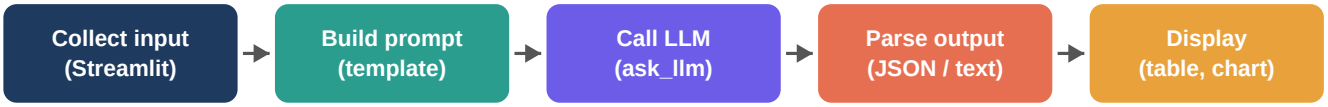


Figure 10: Common pattern behind every project

Project 1: AI Resume Analyzer

Goal: the user uploads a resume and a target role; the app produces extracted skills, skill gaps for that role, improvement suggestions and likely interview questions.

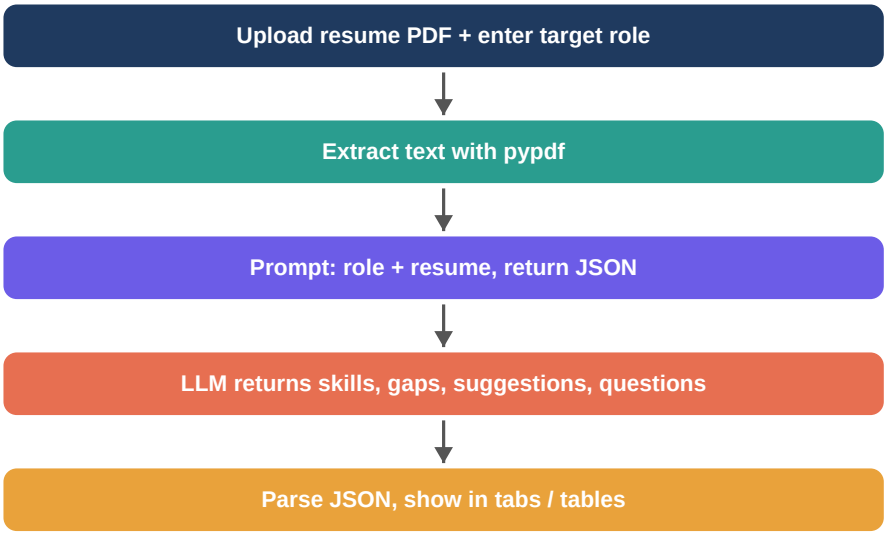


Figure 11: Resume Analyzer flow

Concept: structured JSON output

Asking the model for JSON lets you place each part of the answer in its own UI element (table for skills, list for questions) instead of showing one wall of text.

Key logic

```
PROMPT = """You are an expert technical recruiter.
Analyse the resume for the target role: {role}.
Return ONLY valid JSON with these keys:
  "skills": list of skills found,
  "gaps": list of important missing skills for the role,
  "suggestions": list of at most 5 concrete improvements,
  "interview_questions": list of 8 questions tailored to this resume

Resume:
{resume}"""

raw = ask_llm("You output valid JSON only.", PROMPT.format(role=role, resume=text))
try:
    data = json.loads(raw)
except json.JSONDecodeError:
    st.error("The AI response was not valid JSON. Please try again.")
    st.stop()

t1, t2, t3, t4 = st.tabs(["Skills", "Gaps", "Suggestions", "Interview Questions"])
t1.dataframe(pd.DataFrame({"Skill": data["skills"]}))
t2.write(data["gaps"])
```

Sample output

```
Skills        : Python, SQL, Pandas, Scikit-learn, Git
Gaps          : Docker, cloud deployment (AWS/GCP), model monitoring
Suggestions   : 1) Add quantified results to projects (e.g. 'improved F1 by 8%')
                2) Add a deployed project link   3) Move skills section to top
Interview Qs  : 1) Explain how you handled class imbalance in your churn model.
                2) How would you deploy this model for real-time use?
```

- **Challenge:** add a match score (0-100) and show it with `st.metric`.

Project 2: AI Study Buddy

Goal: a learning assistant that answers questions and generates summaries, MCQs, explanations and study plans from a topic or pasted notes.

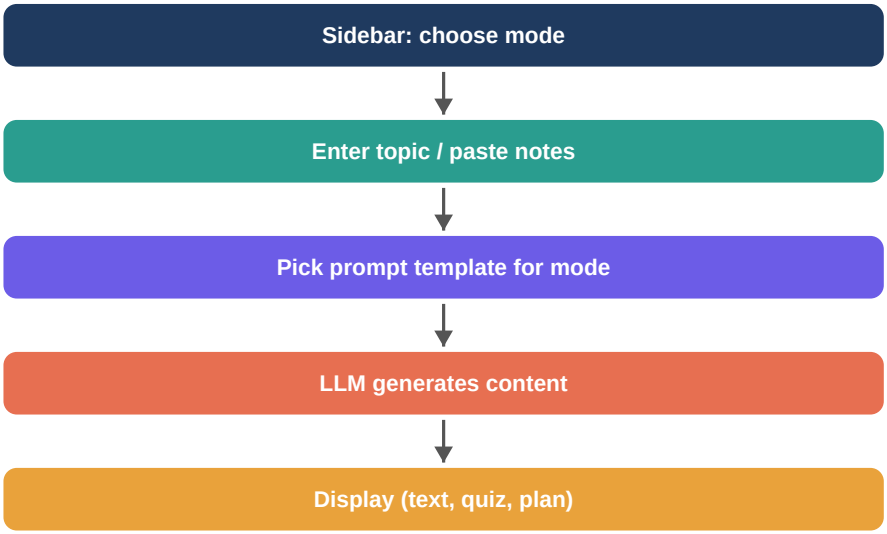


Figure 12: Study Buddy flow

Concept: one app, many prompt templates

Each feature is simply a different prompt over the same input. Store them in a dictionary and let the sidebar choice select which one to use.

Key logic

```
PROMPTS = {
  "Summary":     "Summarise the notes in 5 bullet points for a student:\n{text}",
  "MCQs":        "Create {n} MCQs with 4 options and mark the correct answer, with a "
                 "one-line explanation:\n{text}",
  "Explain":     "Explain this simply, like to a beginner, with one analogy:\n{text}",
  "Study Plan":  "Create a {days}-day study plan with daily goals for:\n{text}",
}

mode = st.sidebar.radio("Mode", list(PROMPTS))
if st.button("Generate"):
    prompt = PROMPTS[mode].format(text=notes, n=5, days=7)
    st.markdown(ask_llm("You are a patient tutor.", prompt))
```

Sample MCQ output

```
Q1. Which layer of a Streamlit AI app should store the API key?
   A) app.py   B) .env file   C) README   D) requirements.txt
   Answer: B  -  Secrets stay outside source code so they are never pushed to GitHub.
```

- **Challenge:** make MCQs interactive using `st.radio` and show a score at the end (store answers in `st.session_state`).

Project 3: AI Interview Coach

Goal: generate technical and HR questions for a role, let the user answer, then give follow-up questions and structured feedback.

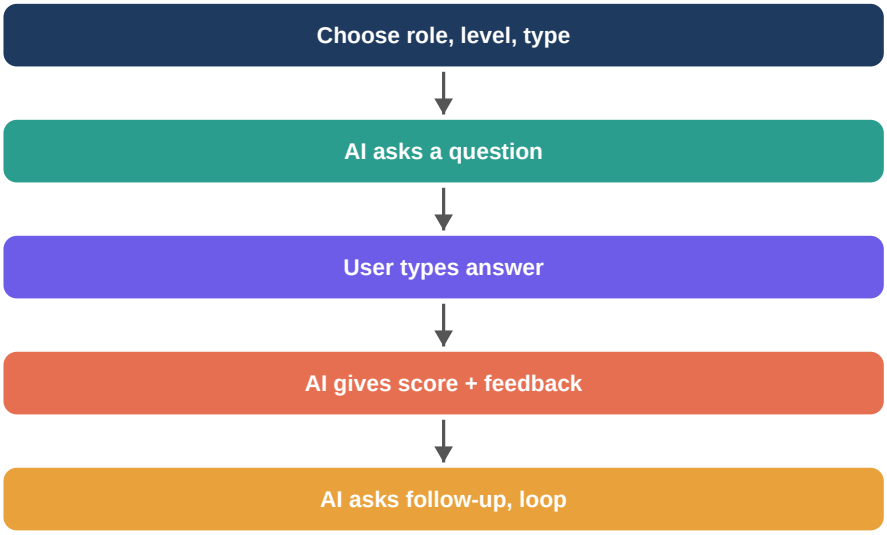


Figure 13: Interview Coach loop (repeats per question)

Concept: stateful conversation

The coach needs to remember the current question and the answers so far. That is exactly what `st.session_state` is for.

Key logic

```
if "question" not in st.session_state:
    st.session_state.question = ask_llm(
        "You are an interviewer.",
        f"Ask ONE {kind} interview question for a {level} {role}. Output only the question.")

st.subheader(st.session_state.question)
answer = st.text_area("Your answer")

if st.button("Get Feedback") and answer:
    feedback = ask_llm("You are a strict but kind interview coach.",
        f"Question: {st.session_state.question}\nAnswer: {answer}\n"
        "Give: Score out of 10, 2 strengths, 2 improvements, a better sample answer, "
        "and one follow-up question.")
    st.markdown(feedback)

if st.button("Next question"):
    del st.session_state["question"]; st.rerun()
```

Sample output

```
Question : Explain the difference between bagging and boosting.
Score    : 6/10
Strengths: Correct definition of bagging; mentioned Random Forest.
Improve  : Explain sequential error correction in boosting; give an example.
Follow-up: When would you prefer boosting over bagging?
```

- **Challenge:** keep a list of all scores and plot progress with `st.line_chart`.

Project 4: AI Expense Advisor

Goal: the user uploads (or enters) expenses; the app analyses spending with pandas, shows charts, and the AI writes budgeting insights.

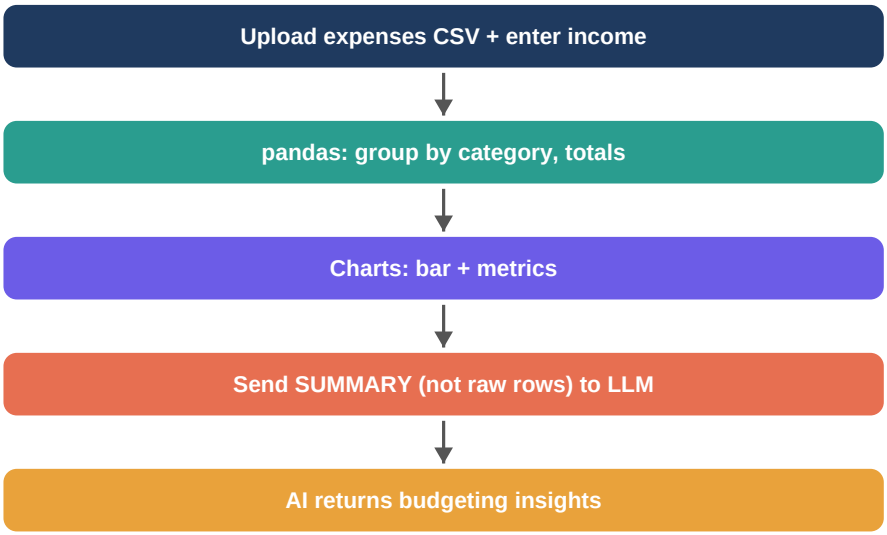


Figure 14: Expense Advisor flow

Concept: let code do the maths, AI do the advice

Models can make arithmetic errors, so compute totals and percentages with pandas and send only the **summary** to the model. This is more accurate, cheaper and protects privacy because raw transaction rows are not sent.

Key logic

```
df = pd.read_csv(file)                                  # Date, Category, Amount
by_cat = df.groupby("Category")["Amount"].sum().sort_values(ascending=False)
total  = by_cat.sum()
share  = (by_cat / total * 100).round(1)

st.metric("Total spent", f"Rs. {total:,.0f}")
st.bar_chart(by_cat)

summary = "\n".join(f"{c}: Rs.{v:,.0f} ({share[c]}%)" for c, v in by_cat.items())
advice = ask_llm("You are a practical personal-finance coach.",
    f"Monthly income: Rs.{income}\nSpending:\n{summary}\n"
    "Compare with the 50/30/20 rule. Give 5 specific, realistic savings tips.")
st.markdown(advice)
```

Sample output

```
Total spent: Rs. 31,500 (income Rs. 40,000)
Rent 12000 (38.1%) | Food 8000 (25.4%) | Travel 5500 (17.5%) | Fun 6000 (19.0%)

Insights:
1. Needs (rent + food) = 63.5% of spending, above the 50% guideline - review food costs.
2. Fun at 19% is high; cap at Rs. 3,000 to free Rs. 3,000 monthly.
3. You save Rs. 8,500 (21%): near the 20% savings target, keep it automatic.
```

> **WARNING**
> AI output is advice, not professional financial guidance. Add a one-line disclaimer in the UI, and avoid asking users to upload sensitive data such as account numbers.

- **Challenge:** add a monthly trend line chart and a 'savings goal' input that the AI uses in its advice.

9. Quick Revision Cheat Sheet

Key terms

Term	One-line definition
Streamlit	Python framework that turns scripts into web apps.
Rerun model	Streamlit re-executes the whole script after every interaction.
session_state	Dictionary that survives reruns; used for chat history and scores.
API key	Secret credential for an AI service; treat like a password.
Environment variable	Named value stored outside code and read at run time.
.env / python-dotenv	File of secrets loaded into the environment on startup.
.gitignore	List of files Git must never track (e.g. .env).
Commit	Saved snapshot of the project with a message.
Remote / origin	The GitHub copy of your repository.
requirements.txt	List of libraries a platform installs before running your app.
Prompt template	Reusable prompt with placeholders for user data.

Command memory aid

Terminal

```
streamlit run app.py                          # run locally
pip install -r requirements.txt               # install dependencies
git init                                      # start repo
git add .  ->  git commit -m "message"        # save snapshot
git remote add origin <url>                   # link GitHub
git push -u origin main                       # upload
```

Self-check questions

- Why does Streamlit rerun the script, and how do you keep a value between reruns?
- Name the four layers of an AI application and what each does.
- Why is hard-coding an API key dangerous, and what two safe alternatives exist?
- What is the order of Git commands from editing a file to seeing it on GitHub?
- Why send a pandas summary to the model instead of raw data in the Expense Advisor?
- What would you test before deploying, and where do the keys go in the cloud?

Final deliverables checklist

Item	Done?
Working Streamlit app with at least 2 input types and clear output	[ ]
AI logic separated from UI code (llm.py / prompts.py)	[ ]
API key in .env / secrets, .env in .gitignore	[ ]
Code pushed to GitHub with clear commit history	[ ]
README with features, tech stack, run steps, screenshot, live link	[ ]
Deployed app link tested from another device	[ ]
5-minute demo rehearsed with backup screenshots	[ ]